SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.Archana	**Reg. No.:**	192425325
Section / Batch:	2024	**Date:**	21-10-2026

Code of Conduct

I __B.Archana(192425325)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____B.Archana_____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Question 1 : Resume Information Extraction Using Regular Expressions

Aim:

To develop a Python-based resume information extraction system using Regular Expressions to extract candidate details, identify technical skills, determine years of experience, generate structured candidate profiles, and shortlist eligible candidates.

Algorithm:

1. Start the program.
2. Import the re module.
3. Store the resume text of multiple candidates.
4. Extract candidate names using Regular Expressions.
5. Identify email addresses and mobile numbers.
6. Detect technical skills such as Python, Java, SQL, Machine Learning, and NLP.
7. Extract years of experience from each resume.
8. Generate a structured summary of each candidate's profile.
9. Check whether the candidate has at least 2 years of experience and Python skill.
10. Display all candidate profiles and eligible candidates.
11. Stop the program.

Python Program:

```
import re
# Sample resume data
resumes = [
    """Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years""",
    """Name: Priya Reddy
Email: priya@gmail.com
Mobile: 9123456780
Skills: Java, SQL, NLP
Experience: 4 years""",
    """Name: Arjun Kumar
Email: arjun@gmail.com
Mobile: 9988776655
Skills: Python, SQL, NLP
Experience: 1 year"""
]
def extract_resume(resume):
    name = re.search(
        r"Name:\s*(.+)", resume, re.IGNORECASE
    )
```

```
    email = re.search(
        r"[\w.-]+@[\w.-]+\.\w+", resume
    )
    mobile = re.search(
        r"\b[6-9]\d{9}\b", resume
    )
    exp = re.search(
        r"Experience:\s*(\d+)\s*years?",
        resume, re.IGNORECASE
    )
    skill_list = [
        "Python", "Java", "SQL",
        "Machine Learning", "NLP"
    ]
    skills = [
        skill for skill in skill_list
        if re.search(
            r"\b" + re.escape(skill) + r"\b",
            resume, re.IGNORECASE
        )
    ]
    years = int(exp.group(1)) if exp else 0

    profile = {
        "Name": name.group(1).strip() if name else "Not Found",
        "Email": email.group(0) if email else "Not Found",
        "Mobile": mobile.group(0) if mobile else "Not Found",
        "Skills": skills,
        "Experience": years
    }
    if years >= 2 and "Python" in skills:
        profile["Eligibility"] = "Eligible"
    else:
        profile["Eligibility"] = "Not Eligible"
    return profile
# Extract candidate information
candidates = [extract_resume(r) for r in resumes]
print("===== CANDIDATE PROFILES =====")
for c in candidates:
    print("\nName:", c["Name"])
    print("Email:", c["Email"])
```

```
    print("Mobile:", c["Mobile"])
    print("Skills:", ", ".join(c["Skills"]) or "None")
    print("Experience:", c["Experience"], "years")
    print("Eligibility:", c["Eligibility"])

print("\n===== ELIGIBLE CANDIDATES =====")

for c in candidates:
    if c["Eligibility"] == "Eligible":
        print(c["Name"])
```

Output:

```
= RESTART: C:/Users/ARCHANA/OneDrive/Pictures/Documents
===== CANDIDATE PROFILES =====

Name: Rahul Sharma
Email: rahul.sharma@gmail.com
Mobile: 9876543210
Technical Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
Eligibility: Eligible

Name: Priya Reddy
Email: priya.reddy@yahoo.com
Mobile: 9123456780
Technical Skills: Java, SQL, NLP
Experience: 4 years
Eligibility: Not Eligible

Name: Arjun Kumar
Email: arjun.kumar@gmail.com
Mobile: 9988776655
Technical Skills: Python, SQL, NLP
Experience: 1 years
Eligibility: Not Eligible

===== ELIGIBLE CANDIDATES =====
Rahul Sharma
```

Result:

Thus, the Python program successfully extracts candidate names, email addresses, mobile numbers, technical skills, and years of experience using Regular Expressions. It generates structured candidate profiles and identifies eligible candidates who have a minimum of 2 years of experience and Python skill.

Question 2 : Product Search System Using Regular Expressions

Aim:

To develop a Python-based product search system using Regular Expressions that supports exact, prefix, suffix, partial, and case-insensitive keyword searches and generates a report showing the matching products and their total count.

Algorithm:

1. Start the program.
2. Import the re module for Regular Expression operations.
3. Create a list containing product names.
4. Accept a search keyword and search type from the user.
5. Use an exact-match pattern to find products matching the complete keyword.
6. Use a prefix pattern to find products beginning with the keyword.
7. Use a suffix pattern to find products ending with the keyword.
8. Use a partial-match pattern to find products containing the keyword.
9. Apply case-insensitive matching using re.IGNORECASE.
10. Display all matching product names.
11. Count the matching products and generate a search report.
12. Stop the program.

Python Program:

```
import re
# List of products
products = [
    "Apple iPhone 15",
    "Samsung Galaxy Phone",
    "Apple Laptop",
    "Wireless Headphones",
    "Laptop Bag",
    "Apple Watch",
    "Gaming Laptop",
    "Bluetooth Speaker",
    "Phone Case",
    "Smart Watch"
]
def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"\b" + re.escape(keyword) + r"\b"
    elif search_type == "prefix":
        pattern = r"^" + re.escape(keyword)
    elif search_type == "suffix":
        pattern = re.escape(keyword) + r"$"
```

```
    elif search_type == "partial":
        pattern = re.escape(keyword)
    else:
        print("Invalid search type!")
        return
    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]
    print("\nSearch Type:", search_type)
    print("Search Keyword:", keyword)

    print("\nMatching Products:")
    if matches:
        for product in matches:
            print("-", product)
    else:
        print("No matching products found.")
    print("Total Matching Products:", len(matches))
    print("-" * 35)
# Perform different searches
search_products("Apple", "exact")
search_products("Apple", "prefix")
search_products("Laptop", "suffix")
search_products("Phone", "partial")
search_products("watch", "partial")
```

Output:

```
Matching Products:
- Apple iPhone 15
- Apple Laptop
- Apple Watch
Total Matching Products: 3
-----------------------------------

Search Type: prefix
Search Keyword: Apple

Matching Products:
- Apple iPhone 15
- Apple Laptop
- Apple Watch
Total Matching Products: 3
-----------------------------------

Search Type: suffix
Search Keyword: Laptop

Matching Products:
- Apple Laptop
- Gaming Laptop
Total Matching Products: 2
```

```
Search Type: partial
Search Keyword: Phone

Matching Products:
- Apple iPhone 15
- Samsung Galaxy Phone
- Wireless Headphones
- Phone Case
Total Matching Products: 4
----------------------------------

Search Type: partial
Search Keyword: watch

Matching Products:
- Apple Watch
- Smart Watch
Total Matching Products: 2
```

Result:

Thus, the Python program successfully implements a product search system using Regular Expressions. It performs exact, prefix, suffix, partial, and case-insensitive searches, displays matching product names, and generates a report containing the total number of matching products for each search.

Question 3: Student Registration Validation System Using Regular Expressions in Python

Aim:

To develop a Python-based student registration validation system using Regular Expressions to validate student details, check the correctness of each field, display appropriate validation messages, and generate a final registration status report.

Algorithm:

1. Start the program.
2. Import the re module.
3. Accept the student register number, institutional email, course code, semester, and mobile number.
4. Define regular expression patterns for each field.
5. Validate the student register number using the specified pattern.
6. Validate the institutional email address and course code.
7. Check whether the semester is within the permitted range.
8. Validate the student's mobile number.
9. Display a success or failure message for each field.
10. Generate the final registration report.
11. If all fields are valid, display “Registration Successful”; otherwise, display “Registration Failed”.
12. Stop the program.

Python Program:

```
import re
# Input student details
register_no = input("Enter Register Number: ").strip()
email = input("Enter Institutional Email: ").strip()
course_code = input("Enter Course Code: ").strip()
semester = input("Enter Semester (1-8): ").strip()
mobile = input("Enter Mobile Number: ").strip()
# Regular expression patterns
register_pattern = r"^[A-Z0-9]{8,12}$"
email_pattern = r"^[a-zA-Z0-9._%+-]+@university\.edu$"
course_pattern = r"^[A-Z]{2,4}\d{3,4}$"
mobile_pattern = r"^[6-9]\d{9}$"
# Validate each field
valid_register = bool(
    re.fullmatch(register_pattern, register_no.upper())
)
valid_email = bool(
    re.fullmatch(email_pattern, email)
)
valid_course = bool(
    re.fullmatch(course_pattern, course_code.upper())
)
valid_semester = (
    semester.isdigit() and 1 <= int(semester) <= 8
)
valid_mobile = bool(
    re.fullmatch(mobile_pattern, mobile)
)
# Display validation results
print("\n===== VALIDATION RESULTS =====")
print("Register Number:",
      "Valid" if valid_register else "Invalid")
print("Institutional Email:",
      "Valid" if valid_email else "Invalid")
print("Course Code:",
      "Valid" if valid_course else "Invalid")
print("Semester:",
      "Valid" if valid_semester else "Invalid")
```

```
print("Mobile Number:",
      "Valid" if valid_mobile else "Invalid")
# Final registration report
print("\n===== REGISTRATION REPORT =====")
if all([
    valid_register,
    valid_email,
    valid_course,
    valid_semester,
    valid_mobile
]):
    print("Registration Successful")
else:
    print("Registration Failed")
    print("Please correct the invalid fields.")
```

Output:

```
= RESTART: C:\Users\ARCHANA\OneDrive\Pictures\Docu
Enter Register Number: 23CS123456
Enter Institutional Email: student@university.edu
Enter Course Code: CSE123
Enter Semester (1-8): 5
Enter Mobile Number: 9876543210

===== VALIDATION RESULTS =====
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

===== REGISTRATION REPORT =====
Registration Successful
```

Result:

Thus, the Python program successfully validates student registration details using Regular Expressions. It checks the register number, institutional email, course code, semester, and mobile number, displays validation messages, and generates a final registration status report.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____